
Django Flatblocks Documentation

Release 4.2.0.dev1

PSU OIT Web & Mobile Team

Jun 28, 2025

Contents

1	django-flatblocks	1
1.1	Installation	1
1.2	Upgrading	1
1.3	Usage	1
1.4	edit-view	2
1.5	History	3

CHAPTER 1

django-flatblocks

django-flatblocks is a simple application for handling small text-blocks on websites. Think about it like `django.contrib.flatpages` just not for a whole page but for only parts of it, like an information text describing what you can do on a site.

1.1 Installation

Probably the easiest way to install this application is to first run `pip install django-flatblocks`. Once this step is complete add "flatblocks" to your `INSTALLED_APPS` setting in your `settings.py` file and run `python manage.py syncdb` to update your database.

1.2 Upgrading

django-flatblocks uses [South](#) for handling data and schema migrations starting with version 0.6.0, so the South-typical update path applies here.

If you're upgrading from a 0.5.x version or earlier you will have to migrate in 3 steps:

1. Install south.
2. Migrate your database to the first version of flatblocks using South:

```
./manage.py migrate flatblocks 0001 --fake
```

3. Then migrate your database to the latest version of flatblocks' database and data structure:

```
./manage.py migrate flatblocks
```

1.3 Usage

Once you've created some instances of the `flatblocks.models.FlatBlock` model, you can load it it using the `flatblocks templatetag-library`:

```
{% load flatblocks %}

<html>
  <head>
    <!-- ... -->
  </head>
  <body>
    <div id="page">
      <div id="main">
        <!-- ... -->
      </div>
      <div id="sidebar">
        {% flatblock "page.info" %}
      </div>
    </div>
  </body>
</html>
```

This way you can display a text block with the name ‘page.info’. If you have the name of a block in a template variable, leave out the quotes.

Additionally you can also specify which template should be used to render the flatblock:

```
{% flatblock "page.info" using="my_template.html" %}
<!-- -->
{% flatblock "page.about" using="my_template.html" %}
```

If you want to simply output the value of the `content` field of a flatblock without using any template, you can use either options:

```
{% flatblock "page.info" using=False %}
```

or

```
{% plain_flatblock "page.info" %}
```

As with the slug of the flatblock also with the template name you have the choice of using the literal name of the template or pass it to the `templatetag` as a variable.

The content of a flatblock (as well as its header) can also be evaluated as a full-fledged Django template:

```
{% flatblock "page.info" evaluated=True %}
```

This also works with the other parameters like the custom template and with the `plain_flatblock` `templatetag`:

```
{% flatblock "page.info" evaluated=True using="my_template.html" %}
<!-- -->
{% plain_flatblock "page.about" evaluated=True %}
```

1.4 edit-view

With `flatblocks.views.edit` django-flatblocks offers a simple view to edit your flatblocks from your frontend. To use it simply include it in your `URLconf` and create a `flatblocks/edit.html` template.

By default the view doesn’t do any permission checking, so you should decorate it accordingly in your `URLconf`:

```

from flatblocks.views import edit
from django.contrib.auth.decorators import login_required

# ...

urlpatterns = pattern('',
    url(r'^flatblocks/(?P<pk>\d+)/edit/$', login_required(edit),
        name='flatblocks-edit'),
    # ...
)

```

The template can operate on following variables:

- form
- flatblock
- origin (the URL of the previous page)

Additionally the view offers some basic customization hooks via these keyword arguments:

template_name

Name of the template to be used for rendering this view. By default `flatblocks/edit.html` is used.

success_url

After successfully editing a flatblock the view will redirect the user to the URL specified here. By default the view will try to determine the last visited page before entering the edit-view (which is normally a page where the flatblock is used) and redirect the user back there.

modelform_class

If you want to use a customized ModelForm class for flatblocks you can specify it here.

permission_check

This argument lets you specify a callback function to do some flatblock-specific permission checking. Such a function could look like this:

```

def my_permcheck(request, flatblock):
    if request.user.is_staff or flatblock.slug == 'free_for_all':
        return True
    return HttpResponseRedirect('/')

```

With this permission callback set, a user that is not a staff-user is not allowed to edit this view unless it's the “free_for_all” block. If these criteria are not met, the user is redirected to the root URL of the page.

The contract here is pretty simple. The permission callback should return `False`, if the user should receive a 403 message when trying to edit this link. If the function returns an instance of `HttpResponse` the view will proceed from the assumption that your view already did everything there is to do and return that response-object. Any other return value tells the view that the permissions are OK for the current user and that it should proceed.

1.5 History

Since this application targets use-cases that are basically applicable to most web-projects out there, there are tons of solutions similar to this one. In fact, this app is a fork originally from [django-chunks](#) developed by Clint Ecker.

In November 2008 Kevin Fricovsky created the [original fork](#) in order to add an additional “active”-flag to each chunk. That project was later on [forked by Peter Baumgardner](#) who removed that flag again and added a “header”-field in order to directly associate and optional title with each text block.

This fork aims now to add more features like variable chunks and also integrate some of the features developed by H. Waara and S. Cranford in the [django-better-chunks](#) fork (`django.contrib.site-` and `i18n-support`).

Unreleased:

- Add support for Django 3.2 and 4.0.
- Add support for Python 3.10.
- Remove support for Django 2.0, 2.2, 3.0, and 3.1.
- Remove support for Python 3.6.
- Change default primary key to use BigAutoField.

1.0.0:

- Add support for Django 2.2, 3.0, and 3.1.
- Add support for Python 3.6, 3.7, 3.8 and 3.9.
- Remove support for Django 1.7, 1.8, 1.9, 1.10, and 1.11.
- Remove support for Python 3.4 and 3.5.
- Move CI to GitHub Actions: <https://github.com/jazzband/django-flatblocks/actions>

0.9.4:

- Drop Python 3.3 support.
- Add support for Django 1.11.

0.9.3:

- Fixed Django 1.10 compatibility

0.9.2:

- Fixed reading of README in setup.py
- Dropped Django 1.4 testing
- Tidied code with flake8 and isort
- Fix support for Django 1.7+
- Fix packaging to exclude tests module

0.9.1:

- Dropped testing of Django 1.5 and 1.6
- Added migrations [Thanks Sergey Fedoseev]

0.9:

NOTE: Major tag syntax changes!

- Modernised to use `simple_tag` and standard kwarg syntax.
- Removed caching - use `{% cache %}` tag instead

0.8:

- Python 3 & Django 1.6 support

0.7:

- Support for evaluated blocks offering access to context variables

0.6:

- South support
- Installation and upgrade instructions

Note: This is primarily a transitional release to get South in here and open this project up for some database changes in the future.

0.5.1

- Removed rendering of the content attribute from the admin list by Michael Fladischer
- PyBabel compatibility by Michael Fladischer
- Fixed caching issue with memcache backend

0.5

- Hungarian translation by Török Gábor
- Method added to demo edit form (#5) by Bill Evans

0.4

- FlatBlock autocreation by Mikhail Korobov (can be enabled/disabled with FLATBLOCKS_AUTOCREATE_STATIC_BLOCKS setting)
- Various fixes by Mikhail Korobov
- Fix by Henrik Heimbuerger for the manifest

0.3.5

- Russian translation by Mikhail Korobov

0.3.4

- Norwegian translation by Eivind Uggedal

0.3.3

- FlatBlock.save should also accept optional kwargs.

0.3.2

- All settings are now in the flatblocks.settings module

0.3.1

- Fixes a bug with FlatBlock.save() failing to reset the cache
- Buildout integration for easier testing
- Example urls.py and flatblocks/edit.html-template

0.3

- createflatblock and deleteflatblock commands
- On saving a flatblock its cache will be cleared
- As last argument of the template tag you can now also specify a template name.

0.2

- Translatable
- `flatblocks.views.edit` view for editing flatblocks

0.1

Initial release